

SISTEMA APRIORI
Venda de Ingressos – Corinthians

PABLO DE SOUSA SANTOS
PEDRO FERNANDES ARAÚJO

1. Requisitos Técnicos e Justificativas

1.1 Linguagem de Programação

Escolha: Python, versão 3.10 ou superior.

O Python foi selecionado por ser a linguagem líder em Ciência de Dados e Inteligência Artificial. Sua sintaxe limpa acelera o desenvolvimento, e o ecossistema oferece bibliotecas prontas para mineração de dados, como a biblioteca Apriori utilizada no protótipo, importada diretamente no topo do código. Além disso, a facilidade de integração via APIs permite que o motor antifraude seja acoplado a sistemas de venda de ingressos já existentes com baixo esforço.

No código, o uso de dataclasses, Enum e tipagem explícita demonstra uma escolha consciente por recursos modernos disponíveis a partir do Python 3.10, como o operador match/case utilizado na função `mostrar_resultado()`.

1.2 Banco de Dados (DB)

Escolha: Híbrido — Relacional + NoSQL em Cache.

Camada	Tecnologia	Finalidade no Projeto
Relacional	PostgreSQL	Armazenar <code>historico_compras</code> , CPF, nome e transações consolidadas com integridade ACID.
Em Memória	Redis (NoSQL)	Armazenar as regras geradas pelo <code>FraudDetector</code> (<code>gerar_regras()</code>) para consulta em milissegundos durante a finalização do pedido.

O algoritmo Apriori executado em `gerar_regras()` é computacionalmente pesado para rodar a cada compra. Por isso, o histórico `historico_compras` fica guardado com segurança no PostgreSQL; um processo em segundo plano,

background job, executa o Apriori periodicamente, salva as regras prontas no Redis e o FraudDetector as consulta de forma instantânea no momento da classificação.

1.3 Infraestrutura Necessária

Escolha: Solução em Nuvem baseada em Containers e Serverless, AWS, Google Cloud ou Azure.

O mercado de ingressos de futebol, especialmente em jogos decisivos do Corinthians, apresenta picos massivos de acesso em curtos intervalos de tempo, abertura de vendas. Um servidor dedicado ou VM estática seriam caros o ano todo e poderiam cair durante a alta demanda.

A arquitetura recomendada combina microsserviços com containers, AWS ECS ou Kubernetes, para os módulos persistentes, API de compras, banco de dados, com funções Serverless, AWS Lambda, para o cálculo do Score de risco, função `calcular_score()` do FraudDetector. Isso permite escalar automaticamente de 0 para 10.000 requisições por segundo e encolher logo em seguida, otimizando custos e garantindo disponibilidade no momento crítico da venda.

2. Fluxograma de Funcionamento

O diagrama abaixo representa o fluxo completo da análise antifraude, do recebimento da compra até a decisão final.

<https://drive.google.com/file/d/1KaKG50bM21jOcl7VL9Q0gd-aAk0blzOC/view?usp=sharing>

Status	Condição (Score)	Ação
BLOQUEADA	Score >= 120	Recusada automaticamente. Alto risco de cambismo ou fraude.
SUSPEITA	60 <= Score < 120	Enviada para mesa de análise humana.
NORMAL	Score < 60	Aprovada automaticamente.

3. Descrição do Fluxo e Requisitos de Execução

3.1 Explicação do Fluxograma

Passo 1 - Entrada de Dados

O fluxo inicia quando o cliente tenta realizar uma compra de ingressos. A função `criar_compra()` coleta os dados via terminal: nome, CPF, quantidade de

ingressos e variáveis comportamentais booleanas, pagamento_pix, compra_madrugada, jogo_final, cpf_recente, cartao_virtual, muitas_tentativas. Esses dados preenchem um objeto Compra, dataclass).

Passo 2 - Validação de Consistência

Em classificar(), a primeira verificação checa se compra.quantidade <= 0. Se verdadeiro, a compra retorna imediatamente com StatusCompra.INVALIDA, sem executar o motor de scoring.

Passo 3 - Mapeamento de Features

O método gerar_transacao() converte as propriedades booleanas da compra em tags textuais, ex.: quantidade >= 6 → "muitos_ingressos"; pagamento_pix = True → "pagamento_pix". Essa lista de tags é o formato que o algoritmo Apriori espera para comparação com as regras.

Passo 4 - Motor Antifraude (FraudDetector)

Ao instanciar FraudDetector(), a função gerar_regras() executa o algoritmo Apriori sobre historico_compras, 15 transações históricas de fraudes conhecidas, gerando regras de associação com suporte mínimo de 20%, confiança mínima de 50% e lift mínimo de 1.0.

Passo 5 - Cálculo Dinâmico de Risco (Scoring)

A função calcular_score() percorre todas as regras geradas. Para cada regra, calcula a interseção entre os seus itens e as features da compra atual. Quando dois ou mais itens coincidem, len(intersecao) >= 2, o score é penalizado com pesos estatísticos: suporte × 100, confiança × 50 e lift × 10.

Passo 6 - Tomada de Decisão

O score consolidado é comparado às constantes LIMITE_BLOQUEIO, 120, e LIMITE_SUSPEITA, 60, definidas no topo do arquivo. A função classificar() retorna um StatusCompra (Enum) e o score numérico. A função mostrar_resultado() exibe o diagnóstico final usando match/case para formatar a mensagem de acordo com cada estado.

3.2 Requisitos Tecnológicos de Execução

Para executar o protótipo localmente, garanta os seguintes pré-requisitos:

Componente	Versão / Referência	Notas
Python	3.10 ou superior	Necessário para o operador match/case utilizado em mostrar_resultado().
apyori	Última versão estável	Biblioteca de regras de associação Apriori. Instalação: pip install apyori
dataclasses / enum	Bibliotecas padrão Python	Nenhuma instalação adicional necessária.

Comando de instalação da dependência:

```
pip install apyori
```

Comando de execução do protótipo:

```
python main.py
```

4. Links do Projeto

Repositório do Código: <https://github.com/P4BLOI/AlgoritmoApriori>